

FLUTTER · INSTALAÇÃO

Como rodar o projeto do zero

Telas de Login e Cadastro — Racha App
Da instalação da IDE até o app na tela

Guia para Windows 10 e 11 · apêndice para macOS e Linux

Testado com Flutter 3.47.0 (stable)

A resposta curta, e como usar este guia

Só abrir o arquivo `.dart` numa IDE não faz o app rodar. Um `.dart` sozinho é um arquivo de texto. Para rodar, ele precisa estar dentro de um **projeto Flutter** — uma pasta com estrutura própria, criada pelo comando `flutter create` — e a máquina precisa ter o **Flutter SDK** instalado.

A boa notícia: a IDE cria esse projeto para você em um clique. O trabalho todo está em instalar o Flutter e configurá-lo **uma única vez** na máquina. Feito isso, abrir qualquer projeto Flutter é imediato.

Escolha o caminho antes de começar

Existem dois, e a diferença entre eles é grande. Como este app **não usa nenhum pacote externo, nem Firebase, nem câmera**, ele roda perfeitamente dentro do Chrome — o que dispensa toda a parte pesada da instalação.

	Caminho rápido — Chrome	Caminho completo — Android
Tempo	20 a 30 minutos	1 a 2 horas
Espaço em disco	~3 GB	~12 GB
O que instala	Git, Flutter e a IDE	Tudo do caminho rápido + Android Studio, Android SDK e emulador
Onde o app abre	Numa janela do Chrome	No emulador ou num celular por USB
Serve para apresentar?	Serve. O código mostrado é exatamente o mesmo	Serve, e tem a aparência de celular de verdade

Recomendação: se o objetivo é só abrir o código e mostrar as telas funcionando, faça o **caminho rápido**. Dá para fazer o caminho completo depois, sem refazer nada — as Partes 1 a 3 servem para os dois.

O que tem neste guia

- **Parte 1** — Instalar o Git
- **Parte 2** — Instalar o Flutter SDK
- **Parte 3** — Configurar a variável de ambiente PATH
- **Parte 4** — Conferir tudo com o `flutter doctor`
- **Parte 5** — Instalar a IDE e os plugins
- **Parte 6** — Criar o projeto e colar o código
- **Parte 7** — Rodar (Chrome, emulador ou celular)
- **Parte 8** — Só para o caminho completo: Android Studio e emulador

- **Parte 9** — Problemas comuns e como resolver

Este guia é para Windows 10 e 11, que é o caso do grupo. No fim há um apêndice com as diferenças para macOS e Linux. Tudo aqui foi testado numa máquina real com Windows 10 e Flutter 3.47.0.

Instalar o Git

O Flutter usa o Git internamente para se atualizar. Sem ele, o Flutter não funciona — então este é o primeiro passo, mesmo para quem nunca usou Git.

1 Baixar

Acesse `git-scm.com/download/win`. O download do instalador de 64 bits começa sozinho.

2 Instalar

Abra o arquivo baixado e vá clicando em **Next** até o fim. **Não é preciso mudar nenhuma opção** — as respostas padrão estão certas para o nosso caso.

3 Conferir

Abra o **Prompt de Comando** (tecla Windows, digite `cmd`, Enter) e rode:

```
git --version
```

Tem que aparecer algo como `git version 2.51.0.windows.1`. Se disser que o comando não foi reconhecido, feche o Prompt, abra de novo e tente outra vez.

Já tem o Git? Muita gente já instalou junto com outra ferramenta. Para descobrir, abra o Prompt de Comando e digite `git --version`. Se aparecer um número de versão, pule esta parte.

Instalar o Flutter SDK

O SDK é o Flutter em si: o compilador, as ferramentas de linha de comando e a biblioteca de widgets. Ele não tem instalador — é um arquivo `.zip` que você extrai numa pasta.

1 Baixar o .zip

Acesse `docs.flutter.dev/get-started/install/windows` e baixe o **Flutter SDK** na versão estável (*stable*). São cerca de 1 GB.

2 Criar a pasta de destino

Abra o Explorador de Arquivos, vá até o disco `C:` e crie uma pasta chamada `flutter`. O caminho final tem que ser exatamente:

```
C:\flutter
```

3 Extrair

Clique com o botão direito no `.zip` baixado → **Extrair tudo** → escolha `C:\` como destino. Ao terminar, confira que existe o arquivo:

```
C:\flutter\bin\flutter.bat
```

Se você acabou com `C:\flutter\flutter\bin`, a pasta ficou duplicada: mova o conteúdo um nível para cima.

O erro mais comum da instalação inteira é escolher a pasta errada. O caminho até o Flutter **não pode** ter:

- espaços — por isso **nunca** em `C:\Program Files\`
- acentos ou cedilha — cuidado com pastas como `Área de Trabalho`
- sincronização do OneDrive — evite `Documentos` e `Desktop`, que costumam estar sincronizados e corrompem o SDK

`C:\flutter` não tem nenhum desses problemas. Use ele.

Configurar a variável de ambiente PATH

O PATH é a lista de pastas onde o Windows procura um programa quando você digita o nome dele no terminal. Sem colocar o Flutter nessa lista, digitar `flutter` devolve “*não é reconhecido como um comando*”.

1 Abrir as configurações

Aperte a tecla **Windows** e digite `variáveis de ambiente`. Clique em **Editar as variáveis de ambiente do sistema**.

2 Entrar na tela certa

Na janela que abrir (*Propriedades do Sistema*), clique no botão **Variáveis de Ambiente...**, embaixo à direita.

3 Achar o Path

A janela tem duas listas. Use a de **cima**, *Variáveis de usuário*. Clique na linha `Path` e depois no botão **Editar...**

Se não existir nenhuma linha `Path` ali, clique em **Novo...** e crie uma com o nome `Path`.

4 Adicionar o Flutter

Clique em **Novo** e digite exatamente:

```
C:\flutter\bin
```

Confirme com **OK** nas três janelas abertas. Não basta fechar no X — precisa ser OK nas três, senão a mudança não é salva.

5 Testar

Feche todos os terminais abertos. Abra um Prompt de Comando novo e rode:

```
flutter --version
```

Deve aparecer a versão, o canal `stable` e o caminho `C:\flutter`. Na primeira vez o comando demora um pouco, porque o Flutter termina de se preparar.

Terminal aberto antes da mudança não enxerga o novo PATH. Se `flutter --version` não funcionar, quase sempre é isso: feche **todas** as janelas de Prompt de Comando, PowerShell e a IDE, e abra de novo. Em casos raros é preciso reiniciar o computador.

Conferir tudo com o flutter doctor

O flutter doctor é uma ferramenta de diagnóstico: ele verifica cada peça da instalação e diz o que está faltando. Vale rodar agora e sempre que algo der errado.

```
flutter doctor
```

A saída é uma lista com [✓] para o que está pronto e [X] ou [!] para o que falta. Assim:

```
[✓] Flutter (Channel stable, 3.47.0, on Microsoft Windows)
[✓] Windows Version (10 Pro 64 bits, 22H2)
[X] Android toolchain - develop for Android devices
[✓] Chrome - develop for the web
[X] Visual Studio - develop Windows apps
[✓] Connected device (2 available)
[✓] Network resources
```

O que precisa estar verde, e o que pode ficar vermelho

Linha	Precisa?
Flutter	Sim, sempre. Se estiver vermelha, revise as Partes 2 e 3.
Chrome	Sim, no caminho rápido. Basta ter o Google Chrome instalado.
Android toolchain	Só no caminho completo. É o que a Parte 8 resolve.
Visual Studio	Não. Serve para gerar app de <i>desktop</i> Windows, que não é o nosso caso. Pode continuar vermelha para sempre.
Connected device	Precisa listar pelo menos um. Chrome já conta.
Network resources	Se falhar, costuma ser antivírus, firewall ou rede de faculdade bloqueando. Tente em outra rede.

Para ver detalhes de cada item, rode `flutter doctor -v`. Ele mostra os caminhos exatos que o Flutter está usando, o que ajuda muito a entender um erro.

Instalar a IDE e os plugins

A IDE é onde o código é escrito e de onde o app é executado. O grupo usa o **IntelliJ IDEA**, então é ele que este guia cobre.

1 Baixar o IntelliJ

Acesse jetbrains.com/idea/download e baixe a edição **Community**, que é gratuita. Ela role para baixo na página — a versão paga (Ultimate) aparece primeiro, mas não é necessária.

2 Instalar

Abra o instalador e siga clicando em **Next**. Na tela de opções, marcar **Add “bin” folder to the PATH** é útil, mas não é obrigatório.

3 Instalar os plugins Flutter e Dart

Abra o IntelliJ. Na tela inicial, clique na engrenagem → **Plugins** (ou *File* → *Settings* → *Plugins*, se já houver um projeto aberto).

Na aba **Marketplace**, busque **Flutter** e clique em **Install**. Ele vai perguntar se quer instalar o **Dart** junto: aceite. Depois clique em **Restart IDE**.

Alternativas, se preferir: o **Android Studio** já é um IntelliJ por dentro e os passos dos plugins são idênticos — e ele é necessário de qualquer forma no caminho completo. O **VS Code** também funciona: instale a extensão **Flutter** e ela puxa a do Dart junto.

Não pule o plugin. Sem o plugin do Flutter, a IDE abre o arquivo e colore o código, mas não oferece o botão de executar nem sabe criar projeto Flutter. É justamente ele que transforma a IDE numa ferramenta de Flutter.

Criar o projeto e colar o código

1 Novo projeto

Na tela inicial do IntelliJ, clique em **New Project**. Na lista da esquerda, escolha **Flutter**.

Se o campo *Flutter SDK path* estiver vazio, preencha com `C:\flutter` e clique em **Next**.

2 Nomear

Em **Project name**, use um nome só com letras minúsculas e `_` — por exemplo `telas_login`. Nome com maiúscula, espaço ou acento é recusado pelo Flutter.

Clique em **Create** e espere. A primeira vez demora alguns minutos, porque o Flutter baixa as dependências do projeto.

3 Colar o código

No painel esquerdo, abra a pasta `lib` e clique duas vezes em `main.dart`. Esse arquivo vem com um app de exemplo (um contador).

Selecione tudo (`Ctrl+A`), apague, e cole o conteúdo do `apresentacao_auth.dart` no lugar. Salve com `Ctrl+S`.

4 Apagar o teste de exemplo

Ainda no painel esquerdo, ache a pasta `test`, clique com o botão direito nela e escolha **Delete**. Confirme.

O passo 4 não é opcional. O `flutter create` gera um teste de exemplo que se refere a uma classe `MyApp` — que existia no código de exemplo, mas não existe no nosso (a nossa classe se chama `RachaApp`). Se esse arquivo ficar, a IDE mostra um erro em vermelho permanente. **O app roda mesmo assim**, mas ninguém quer um erro vermelho na tela na hora de apresentar.

Rodar o app

No alto da janela do IntelliJ, à esquerda do botão verde de play, existe uma caixinha que mostra **onde** o app vai rodar. É ali que se escolhe entre Chrome, emulador e celular.

Opção A — No Chrome (caminho rápido)

Abra a caixinha de dispositivos e escolha **Chrome (web)**. Clique no botão verde de **play**.

Na primeira vez demora cerca de um minuto; depois é rápido. O Chrome abre sozinho com a tela de login do app.

Pelo terminal, o equivalente é:

```
flutter run -d chrome
```

Opção B — No emulador Android

Exige ter feito a Parte 8. Escolha o emulador na caixinha e clique em play. Se o emulador não estiver ligado, o IntelliJ liga sozinho.

Opção C — Num celular Android de verdade

1 Ativar as opções de desenvolvedor

No celular: *Configurações* → *Sobre o telefone* e toque **sete vezes** em **Número da versão**. Aparece a mensagem “Você agora é um desenvolvedor”.

2 Ligar a depuração USB

Configurações → *Sistema* → *Opções do desenvolvedor* e ative **Depuração USB**.

3 Conectar

Ligue o cabo USB no computador. O celular pergunta se autoriza a depuração: marque **Sempre permitir** e confirme. O aparelho aparece na caixinha de dispositivos do IntelliJ.

Se não aparecer, troque o cabo — muitos cabos são só de carga e não transmitem dados.

Recarregar sem reiniciar: com o app rodando, salvar o arquivo (**Ctrl+S**) aplica a mudança na hora, sem perder a tela em que você está. É o *hot reload*, e é ótimo para demonstrar: mude uma cor ou um texto na frente do professor e mostre aparecendo na hora.

Confira antes de apresentar:

- ☐ O app abre direto na tela de login
- ☐ Clicar em “Não tenho conta — cadastrar” abre a tela de cadastro
- ☐ Criar uma conta volta para o login com o email já preenchido
- ☐ Entrar com essa conta leva para a tela inicial com o nome
- ☐ O botão de sair volta para o login
- ☐ Não há nenhum erro em vermelho no painel da IDE

Android Studio, SDK e emulador

Pule esta parte se você escolheu o caminho rápido. Ela só é necessária para rodar o app num emulador ou celular Android. São vários GB de download.

1 Instalar o Android Studio

Baixe em developer.android.com/studio e instale com as opções padrão. Ele traz o Android SDK, que é o que o Flutter precisa — mesmo que você vá programar no IntelliJ.

2 Abrir uma vez

Abra o Android Studio e deixe o assistente inicial terminar de baixar os componentes. **Não pule essa primeira abertura:** é nela que o SDK é de fato instalado.

3 Instalar as ferramentas de linha de comando

No Android Studio: *Settings* → *Languages & Frameworks* → *Android SDK*, aba **SDK Tools**. Marque **Android SDK Command-line Tools (latest)** e aplique.

Sem isso, o `flutter doctor` reclama de `cmdline-tools component is missing`.

4 Aceitar as licenças e criar o emulador

No Prompt de Comando:

```
flutter doctor --android-licenses
```

Responda **y** em todas as perguntas.

Depois, no Android Studio, abra o **Device Manager** (ícone de celular na lateral direita), clique em + → **Create Virtual Device**, escolha um Pixel, baixe a imagem de sistema sugerida e finalize.

O emulador precisa de virtualização ligada. Se ele não iniciar ou ficar travado numa tela preta, quase sempre é isso. A opção fica na BIOS do computador, com nome **Intel VT-x**, **AMD-V** ou **SVM Mode**. Em notebook de faculdade, muitas vezes vem desligada de fábrica.

Emulador pesado demais? Um celular Android ligado por USB (Opção C da Parte 7) funciona melhor e não consome memória do computador. E o Chrome resolve sem nenhum dos dois.

Problemas comuns

Quase tudo que dá errado numa instalação de Flutter está nesta tabela. Quando surgir um erro que não está aqui, o primeiro comando a rodar é sempre `flutter doctor -v`.

O que aparece	O que fazer
<code>'flutter' não é reconhecido como um comando</code>	O PATH não foi configurado, ou o terminal foi aberto antes da mudança. Revise a Parte 3, feche todos os terminais e a IDE, e abra de novo. Confirme também que o arquivo <code>C:\flutter\bin\flutter.bat</code> existe de verdade.
<code>The name 'MyApp' isn't a class</code>	Faltou apagar a pasta <code>test</code> depois de colar o código (Parte 6, passo 4). O app roda mesmo assim, mas o erro fica na tela.
<code>Unable to locate Android SDK</code>	O Android Studio não está instalado, ou foi instalado mas nunca aberto. Abra-o uma vez e deixe o assistente inicial terminar (Parte 8).
<code>cmdline-tools component is missing</code>	Falta marcar Android SDK Command-line Tools no SDK Manager (Parte 8, passo 3).
<code>Android license status unknown</code>	Rode <code>flutter doctor --android-licenses</code> e responda <code>y</code> em tudo.
<code>Visual Studio not installed</code>	Pode ignorar. Só afeta gerar app de desktop Windows, que não é o nosso caso.
<code>No supported devices connected</code>	Nenhum dispositivo foi escolhido. Abra a caixinha ao lado do play e selecione Chrome (web) , ou ligue o emulador.
A IDE não mostra o botão de play	O plugin do Flutter não está instalado ou a IDE não foi reiniciada depois de instalá-lo (Parte 5, passo 3).
O projeto não compila e os erros citam caminhos estranhos	O Flutter foi extraído numa pasta com espaço, acento ou dentro do OneDrive. Mova para <code>C:\flutter</code> e refaça a Parte 3.
Tudo fica muito lento, ou o build trava	Antivírus verificando cada arquivo gerado. Adicione as pastas <code>C:\flutter</code> e a do projeto como exceção no antivírus.
<code>Gradle task assembleDebug failed</code>	Quase sempre é a versão do Java. O Android Studio já traz um Java próprio e o Flutter costuma achá-lo sozinho; se a máquina tiver um Java antigo instalado à parte, ele atrapalha. Rode <code>flutter doctor -v</code> para ver qual Java está sendo usado.
O emulador abre preto ou não inicia	Virtualização desligada na BIOS (Intel VT-x / AMD-V / SVM Mode). Veja o alerta da Parte 8.
Nada funciona e você quer recomeçar	Rode <code>flutter clean</code> dentro da pasta do projeto e depois <code>flutter pub get</code> . Isso apaga o que foi gerado e refaz, sem tocar

O que aparece	O que fazer
	no seu código.

macOS e Linux

Os passos são os mesmos; muda a forma de instalar e de configurar o PATH. Esta parte não foi testada na máquina em que o guia foi escrito — em caso de dúvida, vale seguir a página oficial em `docs.flutter.dev/get-started/install`.

macOS

Com o **Homebrew** instalado, o caminho mais curto é:

```
brew install --cask flutter
```

Sem Homebrew, baixe o `.zip` do site, extraia numa pasta do seu usuário (por exemplo `~/development/flutter`) e acrescente o PATH ao arquivo `~/.zshrc`:

```
export PATH="$PATH:$HOME/development/flutter/bin"
```

Depois rode `source ~/.zshrc` ou abra um terminal novo.

No Mac há uma vantagem: o app também roda no **Safari/Chrome** pelo caminho rápido, e quem tiver o Xcode instalado pode usar o simulador de iPhone.

Linux

Baixe o `.tar.xz` do site oficial, extraia em `~/development/flutter` e adicione ao `~/.bashrc`:

```
export PATH="$PATH:$HOME/development/flutter/bin"
```

Algumas distribuições precisam de pacotes extras antes:

```
sudo apt install git curl unzip xz-utils zip libglu1-mesa
```

Evite instalar o Flutter pelo `snap`: costuma dar problema de permissão com o Android SDK.

Em qualquer sistema, o teste final é o mesmo: `flutter doctor` com a linha do Flutter verde, e pelo menos um dispositivo disponível.